Problem B — An Accelerator for LLM Inference: Hardware and Software

1. The shape of LLM inference

A large language model is a stack of transformer blocks. To produce text from a prompt, the chip runs two phases that look completely different to the silicon underneath. **Prefill** sends the whole prompt through every layer once; the layer is dominated by a few large matrix multiplies of shape roughly $(L \times d) \times (d \times d)$, where L is the prompt length and d is the hidden width. With L in the hundreds, every byte of weight read from memory feeds hundreds of multiply-accumulate operations, so prefill is **compute-bound**: peak FLOPs decide its speed. **Decode** then generates tokens one at a time; each new token re-uses the same weights with $L = 1$, every layer collapses to a matrix × vector, and the chip does only a handful of operations per byte read. Decode is therefore **memory-bandwidth-bound**: GB/s decide its speed, and FLOPs are mostly idle. A 70-billion-parameter model in 8-bit weights occupies 70 GB; generating **one** token requires reading all of it at least once, which on a 3 TB/s HBM stack costs 23 ms before any arithmetic. The bandwidth-bound regime is the one users feel as latency, so we treat it as the primary design constraint.

2. Architectural characteristics

2.1 A memory system designed around the decode roofline

The dominant cost during decode is moving bytes, not computing on them, so the memory subsystem is the most important block on the chip. Two layers matter.

Off-chip — HBM. HBM (High-Bandwidth Memory) is DRAM stacked vertically right next to the compute die and connected through a wide silicon interposer; relative to the DDR DIMMs in a normal server, it delivers 5–10× the bandwidth at far lower energy per byte, because the wires are short and very numerous. A modern accelerator ships with 4–8 stacks giving a total of 3–5 TB/s and 80–200 GB of capacity. **Both numbers matter:** if the model plus its KV-cache (the keys and values produced for every previous token, kept so that attention does not recompute them) does not fit on-package, weights spill to host memory over PCIe and effective bandwidth drops by an order of magnitude. Capacity, not just bandwidth, is therefore a first-class design parameter.

On-chip — SRAM and operator fusion. HBM bytes are precious, so the chip should re-read as little as possible. The lever is on-chip **SRAM** — a small but very fast memory built into the same silicon as the compute units, with nanosecond latency and bandwidth orders of magnitude above HBM. With enough SRAM (tens of MiB per compute cluster) the compiler can **tile** big tensors into blocks that fit on chip and **fuse** several operators into a single pass over those blocks. The canonical case is attention. The naive implementation forms an $L \times L$ score matrix, writes it to HBM, reads it back, softmaxes it, multiplies by V, and writes again — four trips across the bandwidth wall whose total size scales as L^2. If a tile of queries, keys and values fits in SRAM, the score block, the softmax and the value-product all happen on chip, and only the final output tile is written back. This is FlashAttention, and it is the single largest software-visible reason to spend transistors on SRAM: on long contexts it cuts attention bandwidth by more than $10\times$ and turns attention from a memory-bound bottleneck into a compute-bound block. The same tiling logic fuses the GEMM epilogue (bias, RMSNorm, RoPE rotation, KV-cache append) into the preceding matrix multiply, so the activation tensor is never spilled to HBM between layers. SRAM size therefore directly sets how aggressively the compiler can fuse, and how close decode runs to the HBM roof.

2.2 Matrix engines tuned for low precision

Inside each compute tile, the bulk of the work is dense matrix multiplication. The standard structure is a **systolic array**: a 2-D grid of multiply-accumulate cells through which operands flow in lock-step, so each value read from SRAM feeds many multiplications before being discarded. The array's economic value is set almost entirely by its supported numeric formats. Inference, unlike training, tolerates very low precision: there is no backward pass to amplify rounding error, and post-training calibration can absorb most of the remaining loss. In practice, **FP8** (8-bit float, with separate E4M3 and E5M2 variants chosen per-tensor for activations vs. gradients-of-attention) is now the default for both weights and activations, doubling throughput over FP16 with sub-percent accuracy loss, and **INT4 weight-only** quantisation (weights stored in 4 bits, activations kept in FP16/FP8) halves the weight-bandwidth bill on decode — where weight bandwidth is **the** bottleneck — at the cost of a small dequantise step inside the GEMM epilogue. The matrix engine must therefore expose FP16, BF16, FP8 and INT4 as first-class input types with **FP32 accumulation** (a wide running sum keeps the result accurate even when the inputs are narrow). Treating low precision as a fallback rather than a primary path is the most common way for a chip to look fast on paper and slow on real workloads.

2.3 Chip-to-chip interconnect

A 70-B model in 8-bit fits on one chip; a 400-B Mixture-of-Experts model does not, and even a 70-B model is usually sharded across several chips to push decode latency below 50 ms/token. The dominant sharding pattern is **tensor parallelism**: every weight matrix is split column-wise across k chips, and an `all-reduce` collective sums the partial activations after each block. During decode the activation being reduced is tiny (one row of width d, a few KB), so what kills performance is **latency per collective**, not bandwidth. The interconnect must therefore offer (i) sub-microsecond latency between chips in the same node, (ii) of order 1 TB/s of bisection bandwidth to keep prefill all-reduces from becoming the bottleneck, and (iii) hardware support for **overlap**: launching the all-reduce of layer ℓ while the matrix engines start layer $\ell + 1$, so communication and compute share the wall clock instead of stacking. Without it, decode latency stops scaling past two or three chips.

3. Software stack

The stack exists to keep the chip on whichever roof — bandwidth or compute — is binding at the moment. Three layers carry most of the weight.

3.1 A tensor compiler with attention-aware fusion

The frontend (PyTorch, JAX) hands over a static computation graph, which a compiler lowers through an **intermediate representation** (a typed, hardware-neutral description of the graph, similar in spirit to LLVM IR but with tensors as the primitive type) such as MLIR, OpenXLA or TVM. The IR's job on this hardware is concrete: choose tile shapes that match the systolic array and fit the SRAM budget, fuse every element-wise op into the GEMM that produces its input, lower quantised types to the right hardware instruction, and emit the asynchronous DMAs that prefetch the next tile while the current one computes. The frontier feature is **attention-aware fusion**: instead of treating attention as a sequence of generic ops, the compiler recognises the pattern and emits a single kernel templated over head size, sequence length and KV-cache layout (paged or contiguous). The hand-written kernel library still exists — FlashAttention, cuBLAS-style GEMMs, MoE routing — but as a **fallback and a correctness oracle**; on a well-engineered compiler stack the auto-generated path closes most of the gap.

3.2 A serving runtime built around the KV-cache

A trained model becomes a useful product only when many users share one accelerator. The runtime is what makes that sharing efficient, and it is structured entirely around the KV-cache, because the KV-cache is the largest and most awkward tensor in the system: it grows linearly with context length, varies per request, and must persist across decode steps. Two ideas dominate.

Paged KV-cache. Allocating one contiguous slab per request leads to internal fragmentation comparable to early operating systems' contiguous allocation: a request that **might** reach 32 K tokens reserves 32 K tokens' worth of GPU memory from the start, even if it stops at 200. The fix, popularised by vLLM, is to allocate the KV-cache in fixed-size **blocks** (say 16 tokens each) and maintain per-request page tables that map logical token positions to physical blocks. Two requests that share a prompt prefix (a system message, a few-shot template) can then share the prefix blocks by reference, paying the prefill cost only once. The hardware needs nothing more than fast gather/scatter with indirect addressing, but the software win is several-fold: realised throughput typically rises 2--4 × at fixed latency.

Continuous batching. A naive scheduler decodes a batch to completion before admitting new requests; short replies then wait for the longest one in the batch, and the chip is half-idle for most of the window. Continuous batching merges new requests into the running decode batch **every step**, so a request that finishes in 50 tokens frees its slot immediately for an arriving prefill, and a long request keeps decoding without interruption. The runtime must therefore co-schedule prefill and decode in the same step (a **chunked prefill** breaks long prompts into pieces that fit alongside ongoing decodes), respect per-request priorities, and hand the matrix engine a different shape every iteration. **Speculative decoding** sits on top: a small draft model proposes several tokens, the main model verifies them in one prefill-shaped pass, and accepted tokens skip a full decode step — a 2–3× speed-up at no quality loss.

3.3 Quantisation toolchain

The FP8 and INT4 paths in Section 2.2 are only useful if a trained BF16 checkpoint can be lowered into them without retraining. An offline **quantisation toolchain** (GPTQ, AWQ, SmoothQuant) uses a small calibration set — a few hundred prompts — to choose per-channel scales that preserve activation shapes, and an outlier-handling step that keeps a fraction of weight columns in higher precision when their dynamic range demands it. Small in code volume, it is the bridge that makes the chip's headline FP8/INT4 throughput reachable on real models.